

DAY2

울산 4반 이름: 박인우

Chapter 1. 개인 이해 총 정리

실제 서비스는 AI 모델이나 서버 하나를 실행하는 것으로 끝나는 것이 아니라, 핵심 사용자와 문제, 입력 데이터, 제공 결과, 사용 위치, 비용, 보안, 장애 대응까지 포함한 전체 흐름을 먼저 설계해야 한다. 각 구성요소의 책임을 나눈 뒤 적절한 기술을 배치한다.

순수 Java로 HTTP 라우팅, 요청 파싱, 객체 생성과 연결, JDBC DB 처리, BCrypt 비밀번호 해싱, 세션·쿠키, JWT 발급과 검증을 직접 구현. 이를 통해 객체와 메서드의 호출 관계를 이해하고, Spring Boot가 반복적인 서버 설정과 객체 관리를 자동화하는 이유를 체감

클래스별로 각각 파일을 따로 만들어서 관리를 하면 수정이 쉬워짐.

스프링 부트는 개발자가 환경 설정이나 부가코드 작성 대신 순수 비즈니스 로직 개발에만 집중하도록 도움

Chapter 2. 수업 내용 정리

실제 상용 수준의 서비스가 될려면, **ai**추론 서비스를 단순히 띄우는것 뿐만아니라 전체 흐름과 연결을 중시, 어떤위치 어떤사람한테 어떤포지션, 어떤방식으로 쓰일꺼가 설계, 코스트 비용은?

서비스는 [핵심 사용자]가 [현재 겪는 문제]를 해결하도록, [데이터]를 분석해 [구체적인 결과나 행동]을 제공한다. 이 목적을 위해 어떤 아키텍처와 결과를 구성할지 결정, 각 구성요소가 무슨 책임을 맡는지 정한 다음 그 책임에 적합한 기술을 배치

Spring Boot는 **Spring** 기반 서버 개발에 필요한 설정, 객체 생성·연결, 내장 웹 서버 실행, 의존성 관리 등의 반복 작업을 자동화해주는 프레임워크다.

gridle 빌드를하면 클래스파일들과 필요한 것들을 묶어 **.jar**파일로 만들어줌. 그리고 **jar**로 배포

오버로딩 : 같은 이름의 메서드를 매개변수의 개수나 자료형을 다르게 해서 여러개 정의 가능

MVP

→ 최소한 어떤 기능이 있어야 제품 가치가 생기는가?

애자일

→ 그 기능을 짧은 단위로 만들고 검토·개선하는 방법

아키텍처 설계(로그인/가입/인증), 프론트엔드(HTML/CSS/JS), 백엔드(JAVA REST API)

인터페이스에 따라 개발/추상클래스, 컬렉션, 예외처리, db연결

인터페이스는 이기능은 반드시 이런 메서드를 제공해야한다는 계약

solid : 객체지향 코드를 변경하기 쉽고 유지보수하기 좋게 만드는 설계 원칙

객체 지향? : 객체가 어떤 변수들과 어떤 메서드를 가지고있는지 명확하게, 서로 독립적이게 설계

클래스는 객체를 만들기 위한 설계도. 클래스 설계대로 객체가 만들어짐

상속은 부모클래스의 변수및 메서드를 물려받아 중복없이 사용가능

BCrypt 암호화, JWT발급, 인증서버 완성

상속 + 오버라이딩 : 부모 메서드를 자식이 재정의

브라우저에서 회원가입하고, 암호화된 회원 정보를 DB에 저장하고, 로그인하면 JWT를 발급하며, 이후 요청에서 사용자를 인증할 수 있는 순수 Java 인증 서버를 완성하는 것

다형성 = 하나의 인터페이스 타입으로 여러 구현체를 교체해서 사용

교체 용이성, 테스트 용이성

public main 메서드를 시작점으로, 나머지는 전부 객체 생성-> 메서드 호출

한 번의 프로그램 실행에는 명확한 시작점이 필요하고, **main()**에는 객체 생성·연결·서버 시작 정도만 두며 실제 서버 기능은 역할별 클래스로 분리하는 것이 유지보수와 테스트에 유리

패키지 : 클래스들의 논리적 묶음, 경로 선언을 통해 해당 클래스들을 찾을 수 있음

객체 지향이 역할을 분리해서 클래스별로 따로따로 다루기 때문에 파일이 많다.

해시맵 : 장점:의존성없음, 즉시 구현, 단점: 재시작시 데이터 사라짐

JDBC(sql직접작성) vs JPA(객체 db자동 매핑,java메서드 호출로 db조작)

생성자 패턴(필드2-3개, 단순), 정적 팩토리 메서드(생성의미를 명확히 할때), 빌더 패턴(필드 많고 선택적인 경우)

① 객체와 메서드가 서로 어떻게 연결되어 실행되는지 체감하기

② 그걸 순수 **Java**로 직접 해보면서 **Spring Boot**가 왜 필요한지 느끼기

예외(대처가능한 오류)처리 : 미리 사전에 예방, 프로그램이 강제종료되는걸 막음.->발생된 예외를 정상적으로 동작할수있도록 대처하거나 사용자한테 안내문구라도...

이 기능(메서드)은 어떤 상황이 벌어지면 동작하는지 기억해야됨, 어떤 역할을 하는 클래스를 상속받아서 어떤기능을 하는지....

Chapter 3. 궁금했던 점 및 보충

쿠키?

쿠키는 브라우저가 저장하고 요청 때 서버로 보내는 작은 데이터이며, 로그인 서버에서는 주로 세션 ID나 JWT를 담아 사용자를 계속 인증하는 데 사용한다.

인터페이스/추상클래스에 대한 보충 설명

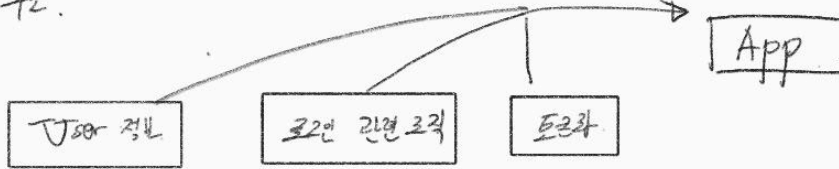
자바는 기본적으로 다중상속 안됨.(상속관계 관리가 안되기 때문) -> 개발자가 구현을 해야할 메서드만 가지고있는 묶음 -> 인터페이스

구분	인터페이스	추상 클래스
핵심 목적	기능 계약 정의	공통 상태·기능 공유
사용 문법	<code>implements</code>	<code>extends</code>
필드	일반 인스턴스 상태 보관에 부적합	인스턴스 필드 가능
생성자	없음	있음
일반 메서드	주로 계약, <code>default/static</code> 가능	자유롭게 구현 가능
추상 메서드	가능	가능
직접 객체 생성	불가능	불가능
한 클래스에서 사용	여러 인터페이스 구현 가능	부모 클래스 하나만 상속 가능

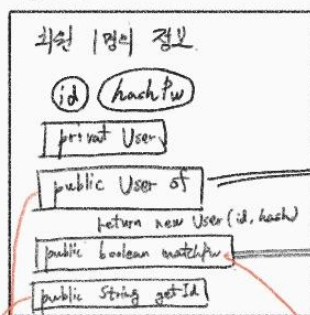
Chapter 4. 코드 이해

DAY 1 보안취약보관

72.



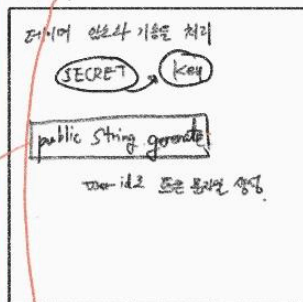
User



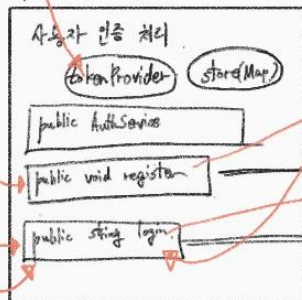
사용자 id를 받아서, 암호화를 해서
그걸로 private User 객체 생성

암호화된 데이터가
입력한 비밀번호와 같은지 비교

TokenProvider



AuthService



사용자 등록

로그인 처리

App

